Creating ASP.NET Core Web API Project using .NET Core CLI

Back to ASP.NET Core Web API Tutorial

Creating ASP.NET Core Web API Project using .NET Core CLI

In this article, I will discuss **how to Create an ASP.NET Core Web API Project using .NET Core** CLI. We will also discuss the different ways to Create ASP.NET Core Web API Project. We will also discuss the MVC project in the default template for the project which is the **Web Application**, and Empty Project, and we will also provide a brief discussion on the **HTTP (Hyperlink Transport Protocol) Protocol**. Let's begin with the discussion.

Creating ASP.NET Core Web API Project

Let's discuss the different ways to Create ASP.NET Core Web API Project. We have two ways to do this.

- **Using .NET Core** Command Line Interface
- **Using Visual Studio**

ASP.NET Core Web API Project using .NET Core CLI

We can create the ASP.NET Core Web API Project using the .NET Core Command Line Interface (CLI) tools. Suppose we want to create the ASP.NET Core Web API Project in the **D:\Demo** Folder. Open a command prompt and navigate to the **D:\Demo** folder. Once you are in the **D:\Demo** folder, then type the following command and press the enter button. This command will create a new folder with the name **MyFirstWebAPIProject** and create all the required files in the folder.

```
D:\Demo>dotnet new webapi -o MyFirstWebAPIProject
```

Let's see the command to create a new ASP.NET Core Empty project. .NET Core CLI. First, type **dotnet new list** and press the enter button as shown in the below image.

```
D:\>dotnet new list
```

Let's see the command to create a new ASP.NET Core Empty project. .NET Core CLI. First, type **dotnet new list** and press the enter button as shown in the below image.

```
D:\>dotnet new list
Usage: new [options]
Options:
  -h, --help        Show help information.
  --debug:reinit    Force the template engine to be reinitialized.
  --dry-run         Displays a summary of what would happen if the new command would create some templates.
  --force           Forces content to be generated even if it would change existing files.
  -l, --list        List templates containing the specified name.
  -n, --name        The name for the output being created.
  -o, --output      Location to place the generated output.
  -lang, --language Filters templates based on language and specifies it for new project.
  --nuget-source    Specifies a NuGet source to use for additional templates.
  -t, --type        Filters templates based on available types. Predefined values are "project", "item" and "other".
  --dry-run         Displays a summary of what would happen if the new command would create some templates.
  --force           Forces content to be generated even if it would change existing files.
  -l, --list        List templates containing the specified name.
  -n, --name        The name for the output being created.
  -o, --output      Location to place the generated output.
  -lang, --language Filters templates based on language and specifies it for new project.
  --nuget-source    Specifies a NuGet source to use for additional templates.
  -t, --type        Filters templates based on available types. Predefined values are "project", "item" and "other".
```

So, you can see in the above image, it provides two template called ASP.NET Core Web API, and to create that project, we can use the command **dotnet new -l webapi** (lowercase letter **l** of the template). The default programming language for the ASP.NET Core Web API is C#, and if we want to change the project, you can also use the language.

Let's create the ASP.NET Core Web API project. We can create a new ASP.NET project using the **dotnet** command. We have two options.

- **dotnet new webapi** -l C# -o **myfirstwebapiproject** (lowercase letter **l** of the template)
- **dotnet new webapi -lang C# -o myfirstwebapiproject** (lowercase letter **l** of the template)

```
D:\>dotnet new webapi -lang C# -o myfirstwebapiproject
The template "ASP.NET Core Web API" was created successfully.
Processing post-creation actions...
Restoring C:\myfirstwebapiproject\myfirstwebapiproject.csproj:
  Determining projects to restore...
  Restored C:\myfirstwebapiproject\myfirstwebapiproject.csproj (in 1.1 sec).
D:\>
```

Let's see the command to create a new ASP.NET Core Empty project. .NET Core CLI. First, type **dotnet new list** and press the enter button as shown in the below image.

```
D:\>dotnet new list
Usage: new [options]
Options:
  -h, --help        Show help information.
  --debug:reinit    Force the template engine to be reinitialized.
  --dry-run         Displays a summary of what would happen if the new command would create some templates.
  --force           Forces content to be generated even if it would change existing files.
  -l, --list        List templates containing the specified name.
  -n, --name        The name for the output being created.
  -o, --output      Location to place the generated output.
  -lang, --language Filters templates based on language and specifies it for new project.
  --nuget-source    Specifies a NuGet source to use for additional templates.
  -t, --type        Filters templates based on available types. Predefined values are "project", "item" and "other".
```

So, you can see in the above image, it provides two template called ASP.NET Core Web API, and to create that project, we can use the command **dotnet new webapi** (lowercase letter **l** of the template). The default programming language for the ASP.NET Core Web API is C#, and if we want to change the project, you can also use the language.

Let's create the ASP.NET Core Web API project. We can create a new ASP.NET project using the **dotnet** command. We have two options.

- **dotnet new webapi** -l C# -o **myfirstwebapiproject** (lowercase letter **l** of the template)
- **dotnet new webapi -lang C# -o myfirstwebapiproject** (lowercase letter **l** of the template)

```
D:\>dotnet new webapi -lang C# -o myfirstwebapiproject
The template "ASP.NET Core Web API" was created successfully.
Processing post-creation actions...
Restoring C:\myfirstwebapiproject\myfirstwebapiproject.csproj:
  Determining projects to restore...
  Restored C:\myfirstwebapiproject\myfirstwebapiproject.csproj (in 1.1 sec).
D:\>
```

Now, navigate to the **D:\Demo** folder. To do so, type the following command and press the enter button.

```
D:\>cd D:\Demo
D:\Demo>
```

Now, type the command **dir** and press the enter button. It will list all the folders and files in the folder.

```
D:\Demo>dir
 Volume in drive D: is Data
 Volume Serial Number is 0A0C-0A0C

 Directory of D:\Demo

05-01-2020  10:00    <DIR>          .
05-01-2020  10:00    <DIR>          ..
05-01-2020  10:00    <DIR>          myfirstwebapiproject
05-01-2020  10:00    <DIR>          MyFirstWebAPIProject
               0 File(s)              0 bytes
               4 Dir(s)  1,000,000,000,000 bytes free
D:\Demo>
```

So you can see in the above image, it provides two folders called **MyFirstWebAPIProject** and **myfirstwebapiproject**. Let's open the folder **MyFirstWebAPIProject**, inside that folder, you can see the following default projects files and folders provided by the Web API template.

```
D:\Demo>cd MyFirstWebAPIProject
D:\Demo\MyFirstWebAPIProject>dir
 Volume in drive D: is Data
 Volume Serial Number is 0A0C-0A0C

 Directory of D:\Demo\MyFirstWebAPIProject

05-01-2020  10:00    <DIR>          .
05-01-2020  10:00    <DIR>          ..
05-01-2020  10:00    <DIR>          Controllers
05-01-2020  10:00    <DIR>          Properties
05-01-2020  10:00             1,337 appsettings.Development.json
05-01-2020  10:00             1,337 appsettings.json
05-01-2020  10:00             1,000 MyFirstWebAPIProject.csproj
05-01-2020  10:00             1,000 MyFirstWebAPIProject.sln
05-01-2020  10:00             4,000 Program.cs
05-01-2020  10:00             4,000 Startup.cs
               6 File(s)         12,674 bytes
               4 Dir(s)  1,000,000,000,000 bytes free
D:\Demo\MyFirstWebAPIProject>
```

Opening ASP.NET Core Web API Project in Visual Studio Code

First of all, you should have installed Visual Studio Code on your machine. Let us see how to open the project in Visual Studio Code. Suppose we want to change the directory to the command prompt to the project directory **D:\MyFirstWebAPIProject**, as shown below.

```
D:\Demo>cd MyFirstWebAPIProject
D:\Demo\MyFirstWebAPIProject>
```

Now, type the command **code .** and press the enter button. It will open the project files in Visual Studio Code, as shown in the image below.

```
D:\Demo\MyFirstWebAPIProject>code .
```

In our upcoming sessions, we will discuss each of the Drives and Folders in detail.

How do you Create ASP.NET Core Web API Project using .NET Core CLI?

Let's see how to build the above created ASP.NET Core Web API Project using .NET Core CLI. We can build the .NET Core Project in two ways.

- **Using Visual Studio Code Terminal**
- **Using Command Prompt**

Build ASP.NET Core Web API Project Using Visual Studio Code Terminal

Open Visual Studio Code. Open the folder **D:\Demo**. Go to, click on the **Terminal** Menu and select the **New Terminal** option from the Visual Studio Code menus, as shown in the image below.

Once you click on the **New Terminal** option, it will open the terminal window, as shown in the below image. In the opening terminal window, type the command **cd MyFirstWebAPIProject** and press the enter button. This command will change the directory to the project directory **MyFirstWebAPIProject**, as shown below.

```
D:\Demo\MyFirstWebAPIProject>
```

Now, type the command **dotnet build** and press the enter button. It will build your project, and you will get the message below.

```
D:\Demo\MyFirstWebAPIProject>dotnet build
Microsoft (R) Build Engine version 16.0.0+ef4ec23d7 for .NET Core
Copyright (C) Microsoft Corporation. All rights reserved.
  Restore completed in 100.7 ms for D:\Demo\MyFirstWebAPIProject\MyFirstWebAPIProject.csproj.
  MyFirstWebAPIProject -> D:\Demo\MyFirstWebAPIProject\bin\Debug\netcoreapp3.1\MyFirstWebAPIProject.dll
Build succeeded.
    0 Warning(s)
    0 Error(s)
Time Elapsed 00:00:02.07
D:\Demo\MyFirstWebAPIProject>
```

How to Run the ASP.NET Core Web API Project using Visual Studio Code?

Now, we are ready to run the ASP.NET Core Web API Project using Visual Studio Code. The .NET Core CLI provides a command to run the ASP.NET Core Web API Application. So, in the terminal, type **dotnet run** and press the enter button, as shown in the below image.

```
D:\Demo\MyFirstWebAPIProject>dotnet run
```

Once you type **dotnet run** and press the enter button, you will get the below message.

```
D:\Demo\MyFirstWebAPIProject>dotnet run
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: https://localhost:5001
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Demo\MyFirstWebAPIProject
```

So, we can see that the port number is 5000. Now, open the browser and type the URL http://localhost:5000/

Now, open the default URL in any of your browsers, and you will get a 404 error.

Click here to see the Solution for this API.

Now, open the **Startup.cs** file in your browsers, and you will get a 404 error.

Click here to see the Solution for this API.

Now, open the **Program.cs** file in your browsers, and you will get a 404 error.

Click here to see the Solution for this API.

Don't worry, just type **swagger** at the end of the URL and press enter, and you will get the following message.

This **swagger** will display the details of all the Web APIs available in your project. As you can see in the above image, it shows one API, i.e. **WeatherForecast**, and the page is fail. Now click on the WeatherForecast API to see the details shown in the image below.

Click here to see the Solution for this API.

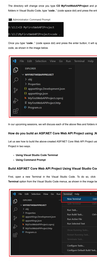
Once you click on the **WeatherForecast** API, it will show you the HTTP details, as shown in the image below.

Click here to see the Solution for this API.

Note: Swagger is also a client API Tool and using Swagger, we can also test the Web APIs. If you are using the default template, then Swagger is by default, swagger is installed into the project.

Test ASP.NET Core Web API using Swagger:

Let's see how to test the ASP.NET Core Web API using Swagger. Open the URL http://localhost:5000/swagger/index.html in the browser.



Click on the **Try it out** button. A pop-up opens, and type the URL http://localhost:5000/swagger/v1/swagger.json in the Parameters section, and click on the **Execute** button.

Once you click on the **Execute** button, it will give you the response, as shown in the image below. Here, you can find the response URL, the response body, the response code, and the response headers.

In our upcoming session, we will discuss where the response is coming from. Even if you want, you can simply call the WeatherForecast API from your browser, and you will get the response in JSON format, as shown in the below image.

In the next article, I will discuss **how to Test ASP.NET Core Web API using Postman**. Here, in this article, I try to explain **How to Create, Build, Run, and Test an ASP.NET Core Web API Project using .NET Core** CLI, and I hope you enjoy this Article **Creating ASP.NET Core Web API Project using .NET Core CLI** article.

Our Site Terms
Please read this updated terms for the 2020 version of our site. It covers certain Pickup Pro Plus and Pickup Pro Premium subscriptions, and it covers the use of our site, our forums, our blogs, our Facebook page, our YouTube channel, our Twitter, Google+, and our Pinterest pages. Our site is owned by Pickup Pro Plus, LLC. Pickup Pro Plus, LLC is a Texas limited liability company.

Share Tweet Pin Mail

Previous Lesson: HTTP (Hyper Text Transport Protocol)
Next Lesson: How to Test ASP.NET Core Web API using Postman

Leave a Reply
Your email address will not be published. Required fields are marked *
Comment *
Name *
Email *
Website
Post Comment